

Investigating Entropy Regularization capability of the Soft-Actor Critic (SAC) Control Algorithm

Rajdeep Adak
radak@asu.edu

ECEE
Arizona State University

Prajjwal Dutta
pdutta9@asu.edu

ECEE
Arizona State University

Abstract — This document provides a summary of our efforts to implement the Soft Actor-Critic (SAC) algorithm to solve the problem of balancing a ball on a planar surface (plate). Unlike other model-free deep reinforcement learning algorithms (off-policy and on-policy based methods), SAC has been shown to provide substantial improvement in terms of performance and sample efficiency. This aspect is important to us since the problem of interest i.e. balancing a ball on a planar surface can have several control strategies. Exploring them using deterministic RL methods can be notoriously expensive [2] in terms of sample complexity due to the high-dimensional observations. So far, we have developed a theoretical grasp of the SAC algorithm and explored its process of implementation on MATLAB. To set up the Reinforcement Learning (RL) environment on MATLAB, we have formulated a 22-dimensional observation vector for a 2-dimensional action space (roll and pitch of the planar surface), with a reward function that awards the RL agent when the ball moves closer to the center of the planar surface, and penalties for absurd plate orientation and excessive control effort. We have built a Simscape multibody model of the plate’s roll and pitch actuator controller that takes torque inputs from the RL agent to balance the ball at the center of the plate. To sense the ball’s position and velocity, we use a 6 DOF joint block. We model the contact force between the ball and the plate using a spatial contact force block. The RL Agent is trained for 6000 episodes with the mean episode reward evaluated every 100th episode until the evaluation score reaches 700. We present an evaluation of the control strategies produced by the fully trained agent. For a given initial condition, the agent noticeably does not generalize to one control strategy. We also investigate how control strategies produced by the trained SAC agent perform when there is a variation in initial conditions.

Keywords— Soft Actor Critic (SAC), Reinforcement Learning (RL), Optimal Control, Off-Policy Learning

I. INTRODUCTION

The Soft Actor-Critic (SAC) [1] is an algorithm that optimizes a stochastic policy in an off-policy manner. The SAC algorithm learns the stochastic policy by maximizing a combination of the policy value and its entropy.

A key aspect of SAC is entropy regularization [3], where the policy is trained to maximize a balance between expected return and entropy, which measures the randomness in the policy. Higher entropy encourages more exploration, potentially

speeding up learning, and helps prevent the policy from prematurely settling into suboptimal local optima. Entropy is a measure of the uncertainty or randomness of a random variable. Let x represent a random variable with a probability mass or density function P . The entropy H of x is calculated based on its distribution P as follows:

$$H(P) = \mathbb{E}_{x \sim P} [-\log P(x)].$$

In entropy-regularized reinforcement learning, the agent gets a bonus reward at each time step proportional to the entropy of the policy at that timestep. This changes the RL problem to:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(R(s_t, a_t, s_{t+1}) + \alpha H(\pi(\cdot | s_t)) \right) \right]$$

II. DYNAMIC ENTROPY REGULARIZATION SAC

There are two variants of SAC that are currently standard: one that uses a fixed entropy regularization coefficient α , and another that enforces an entropy constraint by varying α over the course of training. For this problem, we apply the dynamic entropy regularization that optimizes the entropy from its default value of **0.99** using the Adam optimizer.

The Soft Actor-Critic training algorithm begins by initializing actor (π) and critic (Q_k) networks, along with target critics ($Q_{t,k}$) and an experience buffer. A warm-start phase follows, where the agent interacts with the environment using its initial random policy for NumWarmStartSteps, storing all resulting (S , A , R , S') experience tuples in the buffer.

The primary training cycle then begins. At each timestep, the agent observes state S , selects action A by sampling from its policy $\pi(A|S, \theta)$ (applying bounding for continuous actions), executes A , observes reward R and next state S' , and stores this transition in the experience buffer. Periodically, every D_C steps (LearningFrequency), the agent performs learning updates over several epochs (NumEpoch). In each epoch, it creates up to B (MaxMiniBatchPerEpoch) mini batches, each containing M (miniBatchSize) experiences sampled from the buffer.

Algorithm 1 Soft Actor-Critic (SAC) with Dynamic Entropy Tuning

```

1: Input: Hyperparameters  $(\lambda_Q, \lambda_\pi, \lambda_\alpha, \gamma, \rho, \mathcal{H}_{\text{target}})$ , buffer size, mini-batch size  $M$ , learning frequency  $D_C$ , epochs per update, mini-batches per epoch  $B$ , warm-up steps.
2: Initialize actor  $\pi(\theta)$ , critics  $Q_k(\phi_k)$ , target critics  $Q_{t,k}(\phi_{t,k} \leftarrow \phi_k)$  for  $k = 1, 2$ .
3: Initialize log entropy weight  $\log \alpha$ .
4: Initialize experience buffer  $\mathcal{D}$ .
5: Warm Start: Fill  $\mathcal{D}$  with transitions using an initial policy for ‘NumWarmStartSteps’.
6: repeat (for each time step  $t = 1, 2, \dots$ )
7:   Observe  $S$ , sample action  $A \sim \pi(\cdot|S; \theta)$ .
8:   Execute  $A$ , observe  $R, S'$ . Store  $(S, A, R, S')$  in  $\mathcal{D}$ .
9:   if  $t \pmod{D_C} == 0$  and  $\mathcal{D}$  is sufficiently full then
10:    for each epoch do
11:      for  $j = 1$  to  $B$  do ▷ Iterate through mini-batches
12:        Sample mini-batch  $\{(S_i, A_i, R_i, S'_i)\}_{i=1}^M$  from  $\mathcal{D}$ .
13:        ▷ Compute targets
14:        Sample  $A'_i \sim \pi(\cdot|S'_i; \theta)$ .
15:         $y_i = R_i + \gamma (\min_{k=1,2} Q_{t,k}(S'_i, A'_i; \phi_{t,k}) - \alpha \log \pi(A'_i|S'_i; \theta))$  ▷ Update Critics
16:        for  $k = 1, 2$  do
17:           $\phi_k \leftarrow \phi_k - \lambda_Q \nabla_{\phi_k} \frac{1}{M} \sum_{i=1}^M (Q_k(S_i, A_i; \phi_k) - y_i)^2$ 
18:          ▷ Update Actor
19:          Sample  $\tilde{A}_i \sim \pi(\cdot|S_i; \theta)$  (reparameterized).
20:           $\theta \leftarrow \theta + \lambda_\pi \nabla_{\theta} \frac{1}{M} \sum_{i=1}^M (\min_{k=1,2} Q_k(S_i, \tilde{A}_i; \phi_k) - \alpha \log \pi(\tilde{A}_i|S_i; \theta))$  ▷ Update Entropy Weight  $\alpha$ 
21:          if automatic tuning enabled then
22:             $\mathcal{E}_{\text{batch}} = -\frac{1}{M} \sum_{i=1}^M \log \pi(A_i|S_i; \theta)$ 
23:             $\log \alpha \leftarrow \log \alpha - \lambda_\alpha (\mathcal{E}_{\text{batch}} - \mathcal{H}_{\text{target}})$ 
24:             $\alpha = \exp(\log \alpha)$ 
25:          ▷ Update Target Networks
26:          for  $k = 1, 2$  do
27:             $\phi_{t,k} \leftarrow \rho \phi_{t,k} + (1 - \rho) \phi_k$  ▷ End mini-batch loop
28:          ▷ End epoch loop
29:    until convergence

```

For each mini batch:

1. **Critics (Q_k)** are updated by minimizing the difference between their predictions and target values (y_i). These targets include the reward, the discounted minimum future Q-value from target critics, and the policy entropy term weighted by α .
2. Target Critics ($Q_{t,k}$) are updated slowly towards the main critics using methods like smoothing (TargetsSmoothFactor).
3. **Actor (π)** is updated (controlled by policyUpdateFrequency) to maximize expected Q-values plus weighted policy entropy.
4. **Entropy weight(s) (α)** are adjusted to drive the policy entropy towards a TargetEntropy.

Algorithm 2 Mini-Batch Updates

```

1: Input: Mini-batch  $\{(S_i, A_i, R_i, S'_i)\}_{i=1}^M$ , params  $\phi_k, \phi_{t,k}, \theta$ , weights  $\alpha_d, \alpha_c, \gamma, \mathcal{H}_d, \mathcal{H}_c, K, M, c_{actor}, D_A$ .
2: ▷ 1. Update Critics
3: Compute targets  $y_i$  for  $i = 1..M$ :
4:  $y_i = R_i$  if  $S'_i$  is terminal, else  $y_i = R_i + \gamma (\min_k Q_{t,k}(S'_i, A'_i) - \alpha \times \text{Entropy}(\pi(\cdot|S'_i)))$ 
5: ▷ Note:  $Q_{t,k}$ , Entropy, and  $\alpha$  term depend on action space (discrete/continuous/hybrid)
6: for  $k = 1$  to  $K$  do
7:    $L_k = \frac{1}{M} \sum_{i=1}^M (y_i - Q_k(S_i, A_i; \phi_k))^2$ 
8:    $\phi_k \leftarrow \phi_k - \lambda_Q \nabla_{\phi_k} L_k$  ▷ Update critic  $k$ 
9: ▷ 2. Update Target Critics
10:  $\phi_{t,k} \leftarrow \tau \phi_k + (1 - \tau) \phi_{t,k}$  (or other configured update) for  $k = 1..K$ .
11: Increment  $c_{actor} \leftarrow c_{actor} + 1$ .
12: if  $c_{actor} \pmod{D_A} == 0$  then ▷ 3. Update Actor Entropy Weights (Periodically)
13:   Compute actor objective  $J_\pi = E_{S_i \sim \mathcal{D}, \tilde{A}_i \sim \pi} [\alpha \log \pi(\tilde{A}_i|S_i) - \min_k Q_k(S_i, \tilde{A}_i)]$ 
14:   ▷ Note:  $Q_k$  term depends on action space (uses  $Q_k$  or  $Q_{t,k}$  per original text)
15:    $\theta \leftarrow \theta - \lambda_\pi \nabla_{\theta} J_\pi$  ▷ Update actor
16:   Compute entropy loss  $L_\alpha = E_{S_i \sim \mathcal{D}, A_i \sim \pi} [-\alpha (\log \pi(A_i|S_i) + \mathcal{H})]$ 
17:   Update  $\alpha_d, \alpha_c$  using gradient descent:  $\alpha \leftarrow \alpha - \lambda_\alpha \nabla_{\alpha} L_\alpha$  ▷ Update entropy weights
18:   Reset  $c_{actor} \leftarrow 0$ .

```

III. THE RL MODEL

The following figure shows the RL model containing two subsystems i.e. the RL environment and the RL agent. We chose the `r1SACAgent` as the RL agent object. The RL agent expects observation inputs, reward and episode termination signals from the RL environment.

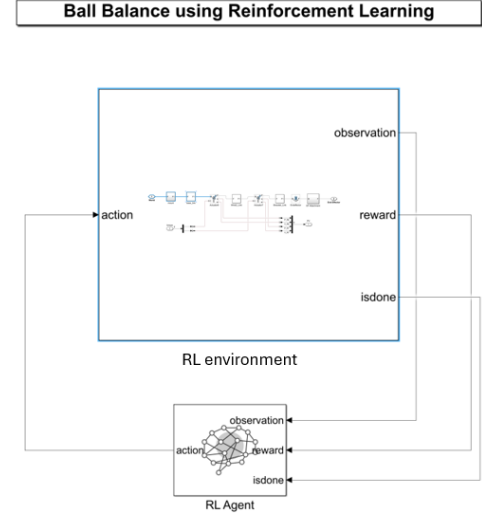


Figure 1: The RL environment and RL agent at a glance

A. The RL Agent

Within the RL agent, we construct the actor and critic agents. We define two critics and use parameterized Q value functions within the critic. We define an actor with a stochastic policy over a continuous action space, implemented by a Gaussian actor. It uses a neural network with two output layers to estimate the mean and standard deviation of the action, with non-negative standard deviations enforced by a ReLU layer. The structure of the actor and critic neural networks are as follows:

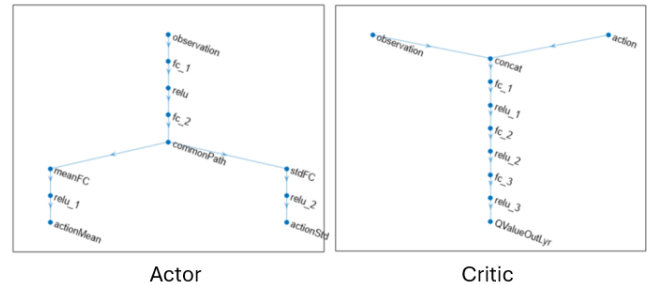


Figure 2: Architecture of the Actor and Critic Neural Networks

B. Continuous Action Generation

For continuous action spaces, the soft actor-critic (SAC) agent utilizes a stochastic actor network that maps a state observation S to the parameters (mean $\mu_\theta(S)$ and standard deviation $\sigma_\theta(S)$) of a Gaussian policy. Action selection proceeds by sampling an unbounded action A from this distribution, $N(\mu_\theta(S), \sigma_\theta(S)^2)$. To satisfy action space constraints, this sample is subsequently transformed, typically via a hyperbolic tangent squashing

function and scaling, yielding the final action A executed in the environment.

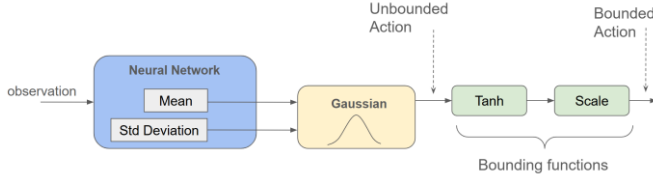


Figure 3: Agent uses the unbounded gaussian distribution to calculate entropy of the policy for a given observation

IV. RL ENVIRONMENT SETUP

The RL environment contains various subsystems. We explain the subsystem components further.

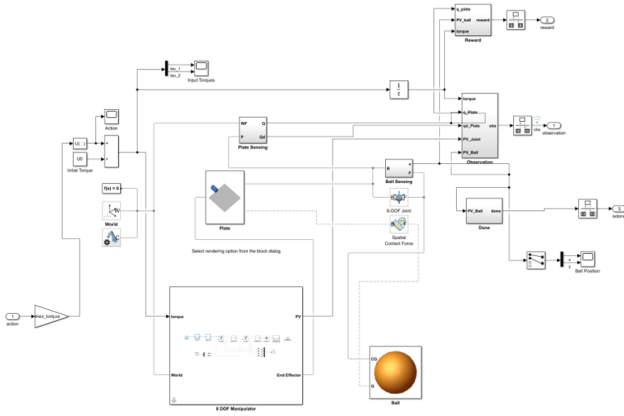


Figure 4: RL Environment model containing the entire control system

A. Ball and Plate Sensing

The **Ball Sensing** block in the Simulink model takes the position and velocity data from the **Ball** and the **Plate**. It then calculates their relative state to output signals representing **Observation**, task completion (**Done**), and performance (**Reward**).

B. Observation Vector

The observation block collects and the dynamic state of a system involving two joints, a ball, and a plate. This data includes joint positions (sine/cosine of angles) and velocities, ball position and velocity, plate orientation (quaternions) and angular velocity, previous joint torques, and ball radius and mass. The joint positions are represented using sine and cosine to avoid angle discontinuities. The plate's orientation is tracked using quaternions, offering a robust representation of 3D rotations. Scaling factors and negative gains are applied to some signals, indicating potential feedback control mechanisms. The block's output aggregates these diverse measurements into a unified representation. The following figure shows the 22-dimensional observation vector.

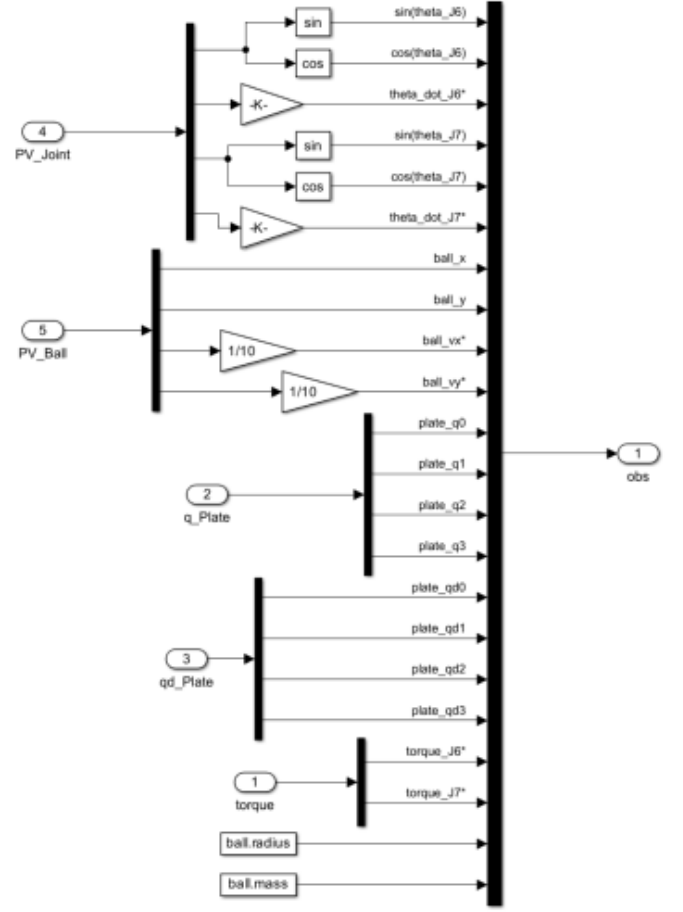


Figure 5: Observation Vector

C. Rewards

The reward block calculates a composite reward signal for an agent controlling a ball-plate system. It incentivizes the agent to move the ball towards the plate's center, maintain plate levelness, and minimize control effort. The ball's position reward is an exponential function of its distance from the center, rewarding proximity.

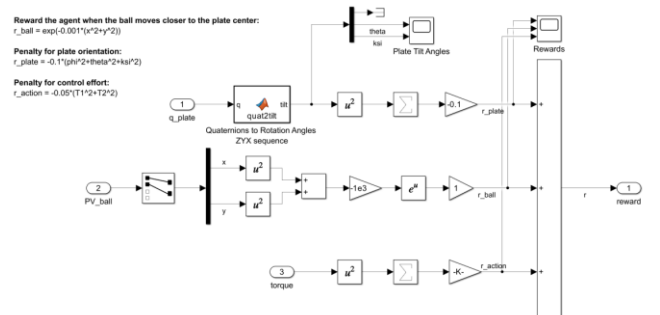


Figure 6: Aggregate Rewards and Penalties

$$r_{\text{ball}} = e^{-0.001(x^2+y^2)}$$

D. Penalties

Plate orientation is penalized based on the squared tilt angles derived from quaternions, discouraging excessive tilting. Control effort is penalized by the squared torques applied, promoting energy efficiency.

$$r_{\text{plate}} = -0.1(\phi^2 + \theta^2 + \psi^2)$$

$$r_{\text{action}} = -0.05(\tau_1^2 + \tau_2^2)$$

Negative scaling factors are used to convert penalties into negative rewards. These individual reward components are summed to generate the final reward signal, guiding the agent towards optimal system behavior.

The reward r_t at time step t is given by:

$$r_t = r_{\text{ball}} + r_{\text{plate}} + r_{\text{action}}$$

E. Plate Roll and Pitch Actuation

To control the roll and pitch of the plate, we intend to use the Simscape model of a 6 DOF robotic arm, but for simplicity we are currently testing the performance of the SAC on a Simscape model of a 2 rotational DOF robot with zero link lengths i.e. the actuation is simulated with 2 cylindrical joints; one rotates about X (roll), and the other rotates about Y (pitch) as shown below.

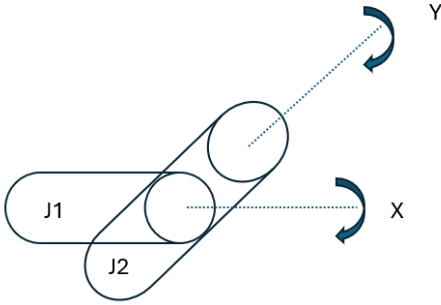


Figure 7: Multibody with two joints J1, J2 controlled by torque generated by the RL agent.

The Simscape model representing the above actuator is as follows:

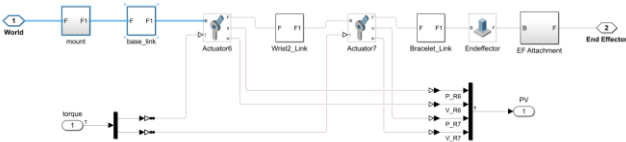


Figure 8: Simscape Multibody containing two joints J1, J2

F. Training and Episode termination

The training process is configured with specific options to control the training behavior. We plan to run each training episode for a maximum of 6000 time steps, corresponding to the `MaxStepsPerEpisode` setting. The agent's performance will be evaluated using the greedy policy, averaging the cumulative reward over 100 consecutive episodes. Training will stop when this averaged reward reaches 700, indicating successful learning. To avoid memory overload, simulation data storage is disabled by setting `SimulationStorageType` to "none". The agent's performance is evaluated periodically every 100 training episodes, using 5 evaluation episodes to assess its progress. This evaluation is handled by the `rlEvaluater` object with the default mean episode reward evaluator. To accelerate training, we utilize parallel computing by setting `trainOpts.UseParallel` to true. Asynchronous parallel workers are employed to further optimize training speed. The training progress is logged using `rlDataLogger`, which tracks the actor and critic losses. This data can be further analyzed and visualized. With the dynamic entropy weight tuning, and parallel computing enabled on an RTX 4090 GPU, the RL agent trained faster. By the 6000th episode, the episode reward reaches near 660. With larger number of episodes, the agent would learn better. The 6000 episodes take nearly 3 hours to complete with the highest reward being 660.48.

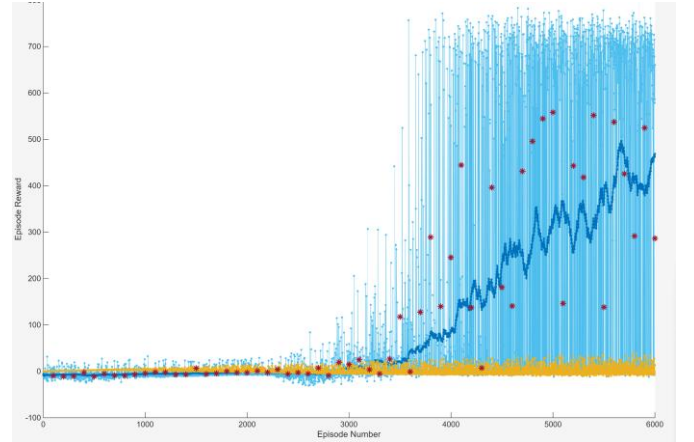


Figure 9: Training results with 6000 episodes

V. RESULTS

A. Actor and Critic Loss

Due to the stochasticity introduced by the gaussian, there is significant noise in the loss plot (see Figure 10). The loss initially increases due to repeated failed control actions for the first few hundred episodes. As the loss becomes consistent the agent begins to learn better control actions from the environment. Further in the training, the loss reduces further indicating that the agent has learned to consistently produce the right control actions.

The critic loss (Figure 11) however does not decrease, but instead has several spikes likely due to critic prediction being significantly different from calculated target (y). This suggests

that the agent encounters some novel situations due to exploration. Secondly, the adam optimizer used to tune the entropy weight also varies over episodes. The nature of the actor-critic training process does not guarantee minimization of critic loss but instead the critic provides a useful learning signal regardless of its own loss minimization.

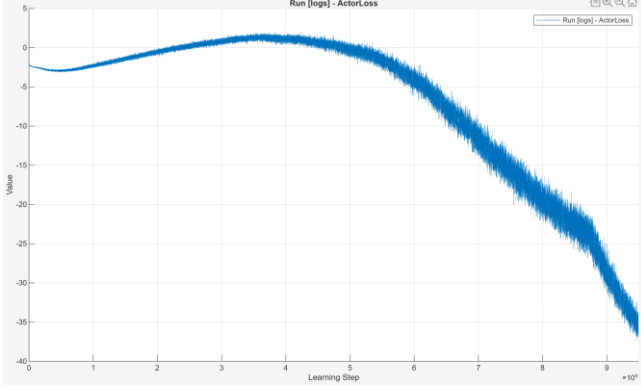


Figure 10: Actor Network Loss

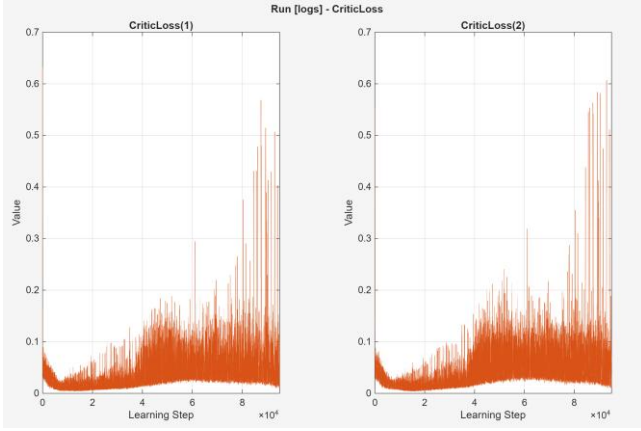


Figure 11: Critic Network Loss

B. RL Agent Performance

The RL agent produces almost no reasonable control actions for the first 3000 episodes. This indicates that the RL agent before learning the right control action, voluntarily explores various ways to fail. This means that SAC typically takes higher training time than other RL alternatives.

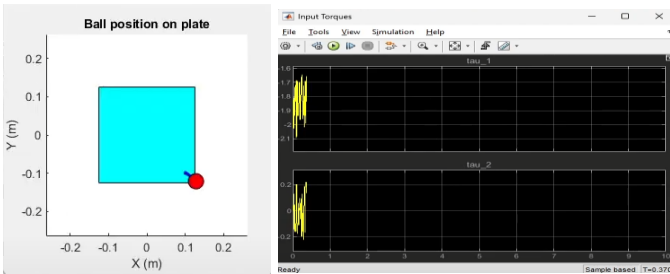


Figure 12: RL performance at 500th episode showing no reasonable control strategy.

Once the episode reward starts increasing, reasonable control strategies can be observed. For an RL agent trained with 6000 episodes, the actuator balances the plate such that the ball can be observed to return to the center. We visualize the path of the ball given a fixed initial condition.

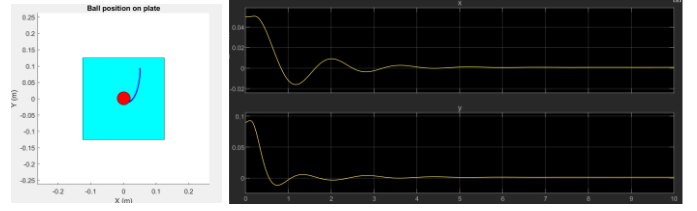


Figure 13: Shows the top view of the plate (left) and the plots of the x and y coordinates of the ball w.r.t the center of the plate

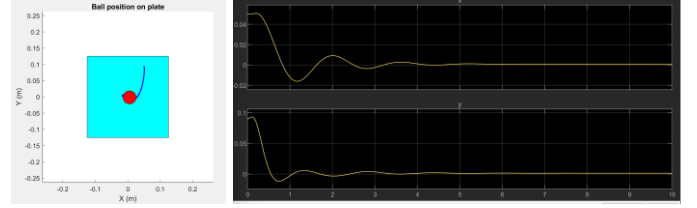


Figure 14: Minor difference in the trajectory of the ball for the same initial condition

Minor difference in the trajectory of the ball is observed when the actuator tries to balance the ball. This indicates that the RL agent has explored yet another approach to balance the ball with given the same initial conditions ($x_0, y_0 = 0.05, 0.09$)

C. RL Agent Performance on different initial conditions

The agent produces a good control strategy for various initial conditions as shown in the figures below.

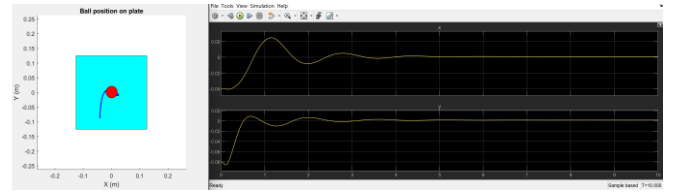


Figure 15: $x_0, y_0 = 0.04, 0.08$

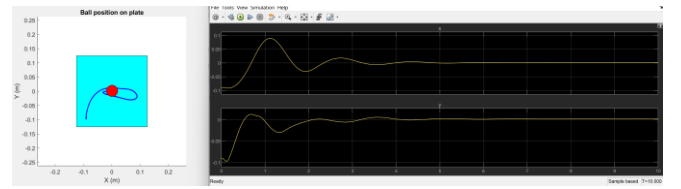


Figure 16: $x_0, y_0 = -0.09, -0.09$

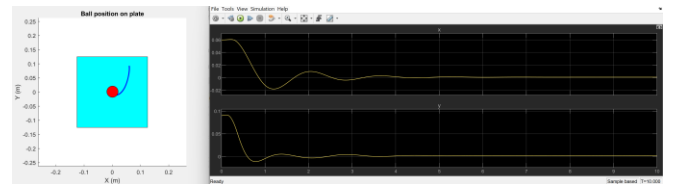


Figure 17: $x_0, y_0 = 0.06, 0.09$

CONCLUSION

A key characteristic of SAC, resulting from its entropy maximization objective ($J(\pi) = E[\sum \gamma^t (R_t + \alpha H(\pi(\cdot|St)))]$), is that it **does not converge to a single deterministic policy**. Instead, the resulting stochastic policy enables the agent to **follow multiple viable trajectories to achieve stabilization**, even from identical initial states. It is likely that with more training episodes, the differences observed for the same initial condition can likely become unnoticeable as the learning reaches an optimal solution. This inherent stochasticity potentially enhances robustness and demonstrates SAC's suitability for continuous state-action control problems.

However, SAC implementation presents limitations. The algorithm is **sensitive to hyperparameters** (e.g., learning rates λ , entropy coefficient α , target update rate τ), requiring careful tuning. Its **sample complexity is often high**, demanding extensive agent-environment interaction data, which impacts simulation time or real-world applicability. Furthermore, **reward function design is complex** and crucial for guiding the agent towards the desired stable behavior without unintended consequences. The **computational cost** of SAC's deep neural networks and optimization also contrasts with potentially simpler classical controllers (e.g., PID, LQR) suitable for this problem. Moreover, reward function design can only be validated iteratively. A good comparison is available on [4].

In summary, SAC is an effective model-free RL method for producing stochastic policies capable of finding diverse solutions. Certainly, there are several applications where SAC can yield results that conventional algorithms may never provide. But its practical application requires careful management of hyperparameter tuning, sample efficiency, reward shaping, and computational resources.

ACKNOWLEDGMENT

We thank **Prof. Jennie Si** for providing valuable guidance and advice during her course on Optimal Control at ASU. The fundamentals of Reinforcement Learning helped us understand concepts involved in SAC. We also thank ASU and the School of Manufacturing Systems and Networks for providing adequate compute and hardware resources. Our ambition from this study is to learn a method of Optimal Control and implement it to solve more complex problems in our upcoming research and in that we are thankful to ECEE for enabling us to take the courses that helped in building the knowledge thus far.

REFERENCES

1. T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor," in *Proc. International Conference on Machine Learning (ICML)*, 2018, vol. 80, pp. 1861–1870. [Online]. Available: <http://proceedings.mlr.press/v80/haarnoja18b.html>
2. T. Haarnoja *et al.*, "Soft Actor-Critic Algorithms and Applications," *arXiv preprint arXiv:1812.05905*, 2018. doi: 10.48550/arXiv.1812.05905. [Online]. Available: <https://arxiv.org/abs/1812.05905>
3. Y. Zhou, J. Lin, S. Wang, and C. Zhang, "Learning Ball-balancing Robot Through Deep Reinforcement Learning," in *Proc. 2020 3rd Int. Conf. Robot., Control Autom. Eng. (RCAE)*, 2020, pp. 5-9. doi: 10.1109/RCAE51096.2020.9279296.
4. MathWorks, "Create Reinforcement Learning Agents," MathWorks Help Center. Accessed: Apr. 21, 2025. [Online]. Available: <https://www.mathworks.com/help/reinforcement-learning/ug/create-agents-for-reinforcement-learning.html>